

**BE PROGRAMMATION
IMPERATIVE FORTRAN**

**ADELE LE NEINDRE ET YVAN
JACOB**

ENSEEIH

Année 2025-2026

PROBLEMATIQUE

Une collectivité locale accueille sur son territoire une industrie potentiellement polluante qui, en cas d'accident, pourrait rejeter dans le cours d'eau qui la traverse un polluant de concentration C_0 .



But : concevoir un outil numérique permettant de modéliser le transport de ce polluant

PROBLEMATIQUE

Une collectivité locale accueille sur son territoire une industrie potentiellement polluante qui, en cas d'accident, pourrait rejeter dans le cours d'eau qui la traverse un polluant de concentration C_0 .



But : concevoir un outil numérique permettant de modéliser le transport de ce polluant

MODELISATION

Objectif : déterminer la concentration C à t_f

Protocole : résoudre l'équation différentielle

$$\boxed{\frac{\partial C(x,t)}{\partial t} + u \frac{\partial C(x,t)}{\partial x} = 0}$$

**vitesse dans tout
le domaine**



CONDITIONS LIMITES

à t = 0:

$$\mathbf{C(x,0) = C_0 f(x) \quad L \leq x \leq 0 \text{ et}} \\ \mathbf{f(x) = H(x-L/10) - H(x-L/5)}$$

$$\mathbf{C(0,t) = C(L,t)}$$

METHODE DES DIFFERENCES FINIES

- 1) discrétisation du domaine de longueur L et de direction x en N_x-1 segment de longueur Δx
- 2) solutions en temps calculées tous les Δt (discrétisation du domaine temporel en N_t-1 points de 0 à t_f)

Méthode: remplacer les dérivées temporelle et spatiale par les approximations :

$$\begin{aligned}\frac{\partial C(x,t)}{\partial t} &= \frac{C(x_k, t + \Delta t) - C(x_k, t)}{\Delta t} \\ \frac{\partial C(x,t)}{\partial x} &= \frac{C(x_k, t) - C(x_k - \Delta x, t)}{\Delta x}\end{aligned}$$



$$\frac{C(x_k, t + \Delta t) - C(x_k, t)}{\Delta t} + u \frac{C(x_k, t) - C(x_k - \Delta x, t)}{\Delta x} = 0$$

RESOLUTION

ETAPE 1: Créer un programme qui:

- récupère les données d'entrée du problème dans un fichier de données d'entrée.

RESOLUTION

```
SUBROUTINE read_data(get,recup)
  USE m_type
  IMPLICIT NONE
  TYPE(phys), INTENT(OUT) :: recup
  TYPE(num), INTENT(OUT) :: get

  OPEN(10,FILE="donnees.dat")
  READ(10,*) recup%L
  READ(10,*) recup%U
  READ(10,*) recup%C0
  READ(10,*) recup%tf
  READ(10,*)
  READ(10,*)
  READ(10,*)
  READ(10,*) get%Nx
  READ(10,*) get%Nt
  CLOSE(10)
END SUBROUTINE read_data
```

RESOLUTION

ETAPE 1: Créer un programme qui:

- récupère les données d'entrée du problème dans un fichier de données d'entrée.
- créer un maillage régulier.

$$x_{reg}(0)=0$$

$$x_{reg}(i)=\frac{L(i-1)}{Nx-1}, 1 \leq i \leq Nx$$

$$t_{reg}(0)=0$$

$$t_{reg}(j)=\frac{tf(j-1)}{Nt-1}, 1 \leq j \leq Nt$$

RESOLUTION

```
SUBROUTINE mesh(x_reg,t_reg, get,recup)
  USE m_type
  IMPLICIT NONE
  TYPE(phys), INTENT(IN) :: recup
  TYPE(num), INTENT(IN) :: get

  REAL, DIMENSION(get%Nx), INTENT(OUT) :: x_reg
  REAL, DIMENSION(get%Nt), INTENT(OUT) :: t_reg
  INTEGER :: i,j

  DO i=1,get%Nx
    x_reg(i)=recup%L*real(i-1)/real(get%Nx-1)
  END DO
  DO j=1,get%Nt
    t_reg(j)=recup%tf*real(j-1)/real(get%Nt-1)
  END DO

END SUBROUTINE mesh
```

RESOLUTION

ETAPE 1: Créer un programme qui:

- récupère les données d'entrée du problème dans un fichier de données d'entrée.
- créer un maillage régulier.
- initialise la concentration dans le milieu à $t=0$.

RESOLUTION

```
FUNCTION Cinit(x,get,recup,H)
  USE m_type
  IMPLICIT NONE
  REAL, INTENT(IN) :: x
  REAL :: Cinit
  REAL :: H
  TYPE(phys), INTENT(IN) :: recup
  TYPE(num), INTENT(IN) :: get

  PRINT *, 'DEBUG Cinit: x=', x, 'borne1=', (recup%L)/10, 'borne2=', (2*recup%L)/10
  PRINT *, 'DEBUG Cinit: H1=', H(x-(recup%L)/10), 'H2=', H(x-(2*recup%L)/10), 'H1-H2=', (H(x-(recup%L)/10)-H(x-(2*recup%L)/10))

  Cinit = recup%C0*(H(x-(recup%L)/10)-H(x-(2*recup%L)/10))

END FUNCTION Cinit
```

```
FUNCTION H(x)
  IMPLICIT NONE
  REAL, INTENT(IN) :: x
  REAL :: H

  IF (x < 0) THEN
    H = 0.0
  ELSE
    H = 1.0
  END IF
END FUNCTION H
```

RESOLUTION

ETAPE 1: Créer un programme qui:

- récupère les données d'entrée du problème dans un fichier de données d'entrée.
- créer un maillage régulier.
- initialise la concentration dans le milieu à $t=0$.
- permet de passer de l'étape t à l'étape $t + \Delta t$.

Objectif : calculer le vecteur à l'instant $t + dt$

Exprimer $C(x, t + dt)$ fonction de $C(x, t)$

$$\left\{ \begin{array}{l} C(x_k, t + \Delta t) = C(x_N, t + \Delta t) \quad \mathbf{k} = \mathbf{1} \\ \\ C(x_k, t + \Delta t) = C(x_k, t) - u \frac{\Delta t}{\Delta x} \cdot (C(x_k, t) - C(x_k - \Delta x, t)) \quad \mathbf{1} < \mathbf{k} \leq \mathbf{N} \end{array} \right.$$

RESOLUTION

```
SUBROUTINE tempo(t_reg, x_reg, CT, CDT, get, recup)
  USE m_type
  IMPLICIT NONE

  TYPE(phys), INTENT(IN) :: recup
  TYPE(num), INTENT(IN) :: get
  REAL, DIMENSION(get%Nx), INTENT(IN) :: CT
  REAL, DIMENSION(get%Nx), INTENT(OUT) :: CDT
  REAL, DIMENSION(get%Nx), INTENT(IN) :: x_reg
  REAL, DIMENSION(get%Nt), INTENT(IN) :: t_reg
  INTEGER :: i

  CDT(1) = CT(get%Nx)

  DO i = 2, get%Nx
    CDT(i) = CT(i) - recup%U * (recup%tf/REAL(get%Nt)) / (recup%L/REAL(get%Nx)) * (CT(i) - CT(i-1))
  END DO

END SUBROUTINE tempo
```

RESOLUTION

ETAPE 1: Créer un programme qui:

- récupère les données d'entrée du problème dans un fichier de données d'entrée.
- créer un maillage régulier.
- initialise la concentration dans le milieu à $t=0$.
- permet de passer de l'étape t à l'étape $t + \Delta t$.
- écrit à des temps réguliers la distribution spatiale de la concentration dans le domaine dans un fichier de résultats.

RESOLUTION

```
SUBROUTINE spat(t_reg, x_reg, CT, CDT, get, recup)
  USE m_type
  IMPLICIT NONE

  TYPE(phys), INTENT(IN) :: recup
  TYPE(num), INTENT(IN) :: get
  REAL, DIMENSION(get%Nx), INTENT(OUT) :: CT, CDT
  REAL, DIMENSION(get%Nx), INTENT(IN) :: x_reg
  REAL, DIMENSION(get%Nt), INTENT(IN) :: t_reg

  REAL, DIMENSION(get%Nx) :: Cpremier, Creset
  INTEGER :: j, p
  REAL :: Cinit, H

  DO p = 1, get%Nx
    Cpremier(p) = Cinit(x_reg(p), get, recup, H)
    ! PRINT *, 'x=', x_reg(p), 'Cinit=', Cpremier(p)
  END DO
  PRINT *, 'Profil initial complet:', Cpremier

  Creset(:) = 0.0
  CDT(:) = 0.0

  OPEN(11, FILE="sortie.txt")

  DO j = 2, get%Nt
    WRITE(11, *) "temps = ", t_reg(j)
    CALL tempo(t_reg, x_reg, Cpremier, CDT, get, recup)
    WRITE(11, *) Cpremier

    Cpremier = CDT
    CDT = Creset
  END DO
  CLOSE(11)
  PRINT *, 'Simulation terminée - fichier sortie.txt écrit'
  CT = Cpremier

END SUBROUTINE spat
```

RESOLUTION

PROGRAMME

```
PROGRAM BE_ETAPE_1
  USE m_type
  IMPLICIT NONE

  TYPE(phys) :: recup
  TYPE(num) :: get
  REAL, DIMENSION (:), ALLOCATABLE :: CT, CDT
  REAL, DIMENSION (:), ALLOCATABLE :: x_reg, t_reg

  CALL read_data(get,recup)

  PRINT *, 'Nx =', get%Nx, 'Nt =', get%Nt
  PRINT *, 'L =', recup%L, 'U =', recup%U, 'C0 =', recup%C0, 'tf =', recup%tf

  IF (get%Nx <= 0 .OR. get%Nt <= 0) THEN
    PRINT *, 'ERREUR: Nx ou Nt <= 0'
    STOP
  END IF

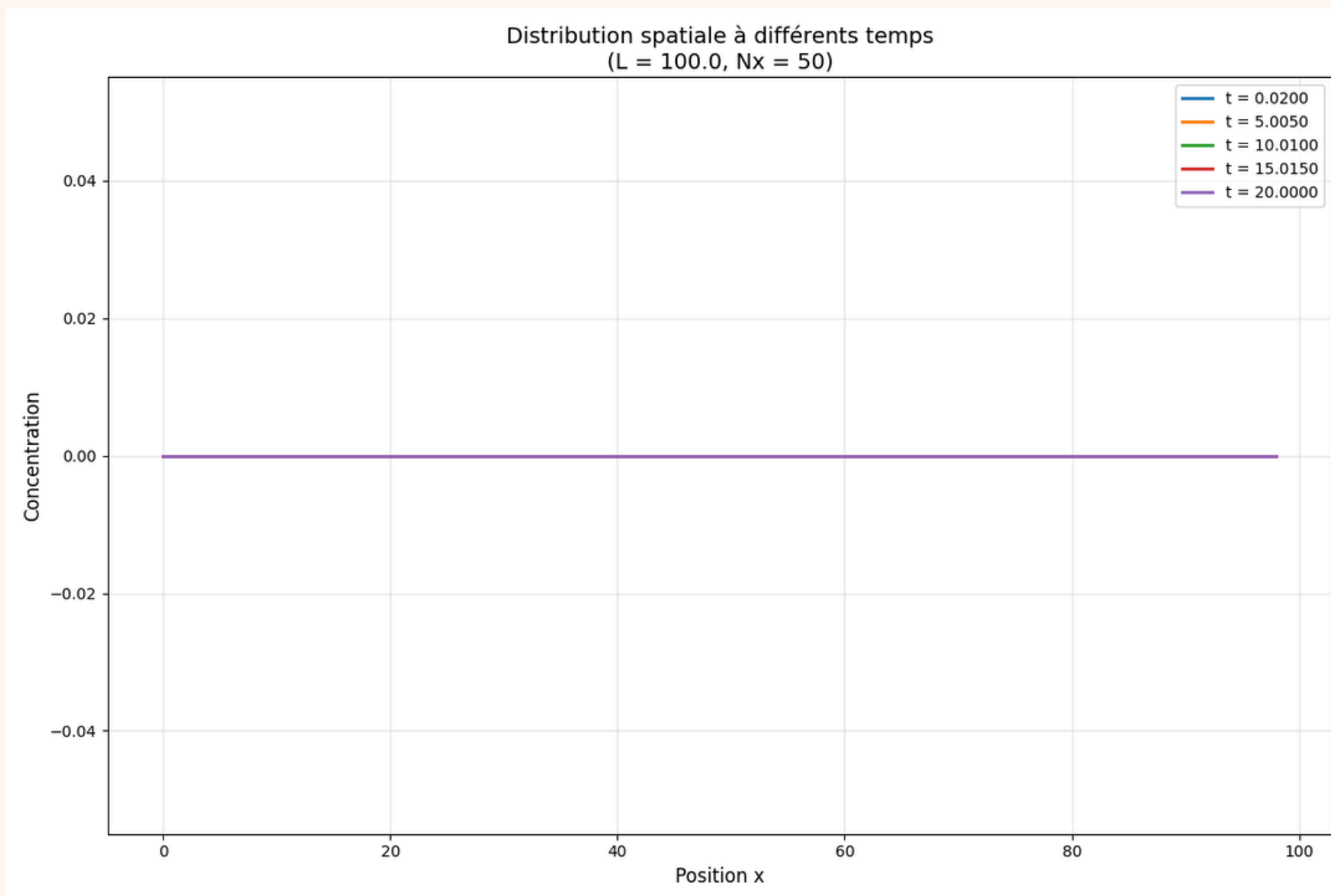
  ALLOCATE(CT(get%Nx), CDT(get%Nx), t_reg(get%Nt), x_reg(get%Nx))
  CT = 0.0
  CDT = 0.0

  CALL mesh(x_reg, t_reg, get, recup)
  ! PRINT *, 'Mesh créé avec succès'
  CALL spat(t_reg, x_reg, CT, CDT, get, recup)
  DEALLOCATE(CT, CDT, t_reg, x_reg)
END PROGRAM BE_ETAPE_1
```

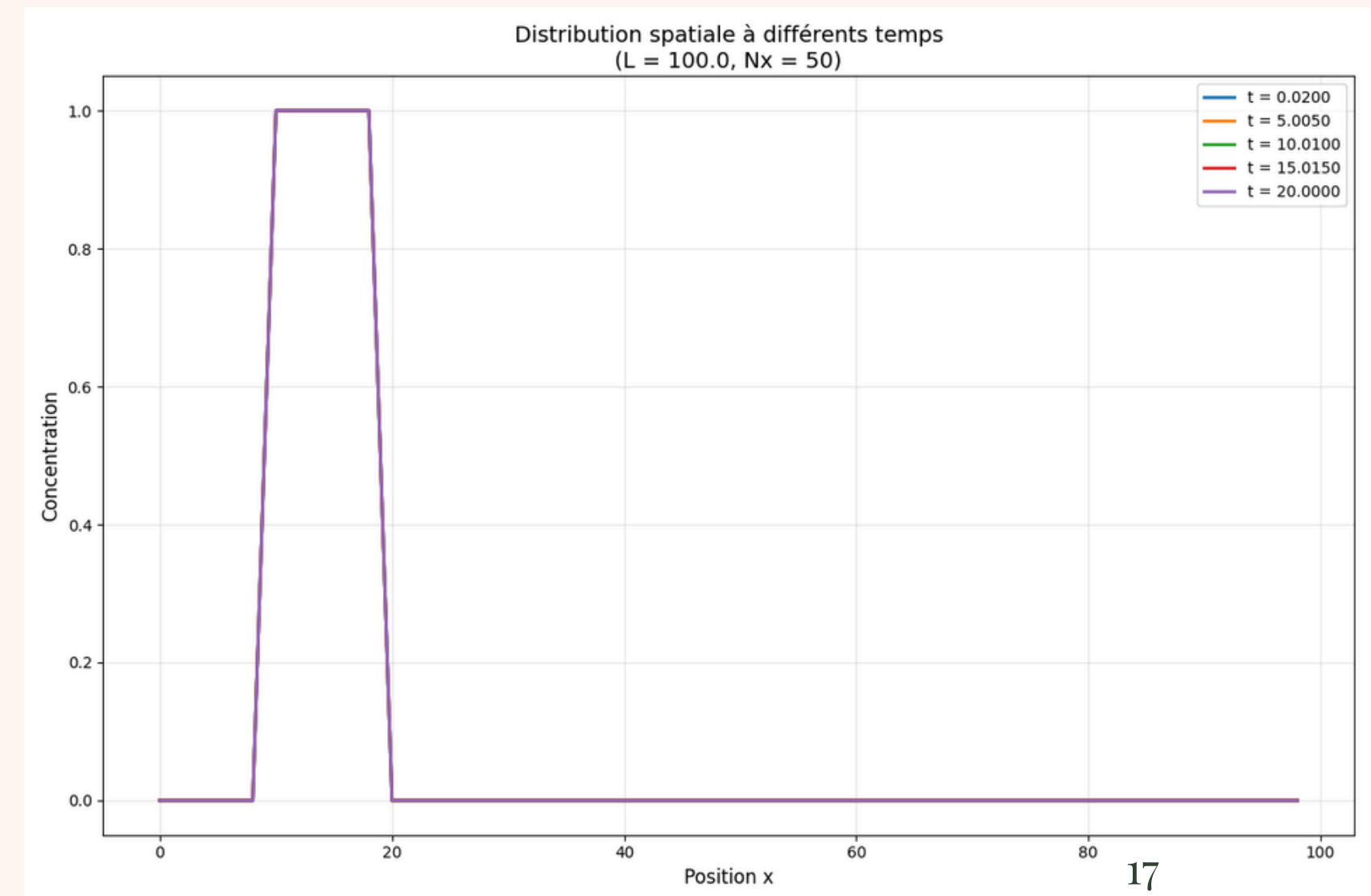
RESOLUTION

ETAPE 2: Vérification du programme

$C_0 = 0$ et $u = 1$



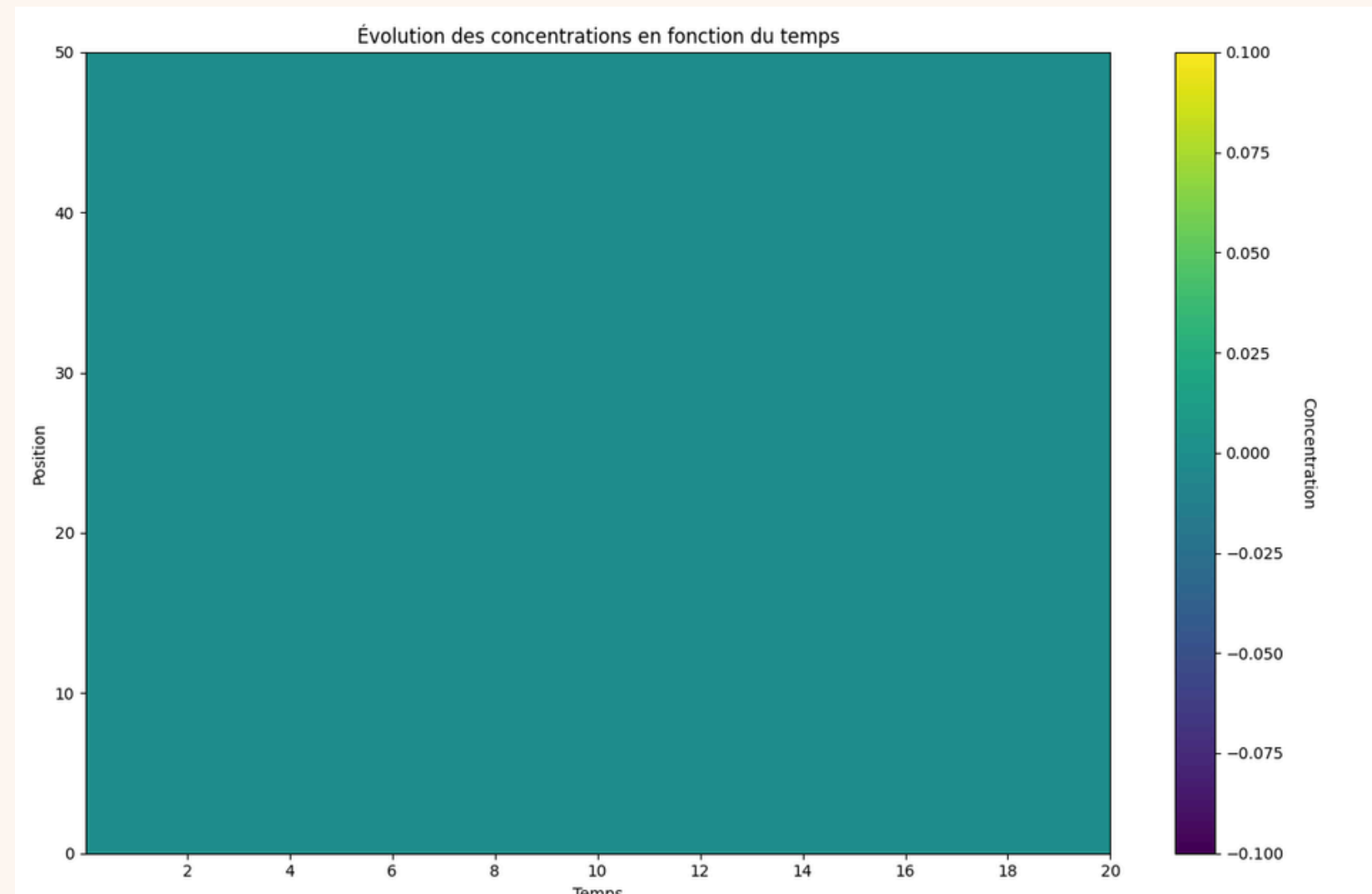
$C_0 = 1$ et $u = 0$



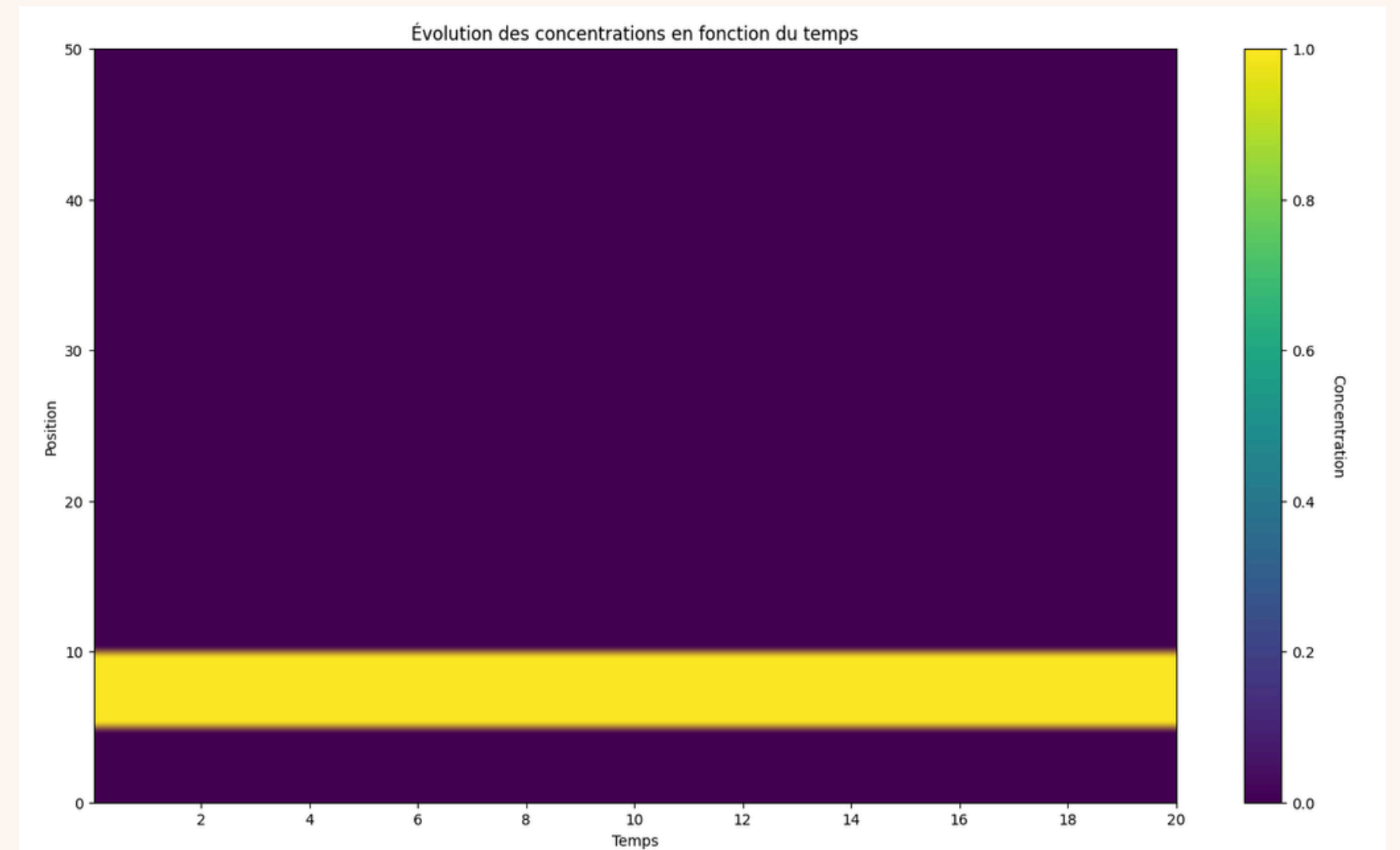
RESOLUTION

ETAPE 2: Vérification du programme

$C_0 = 0$ et $u = 1$



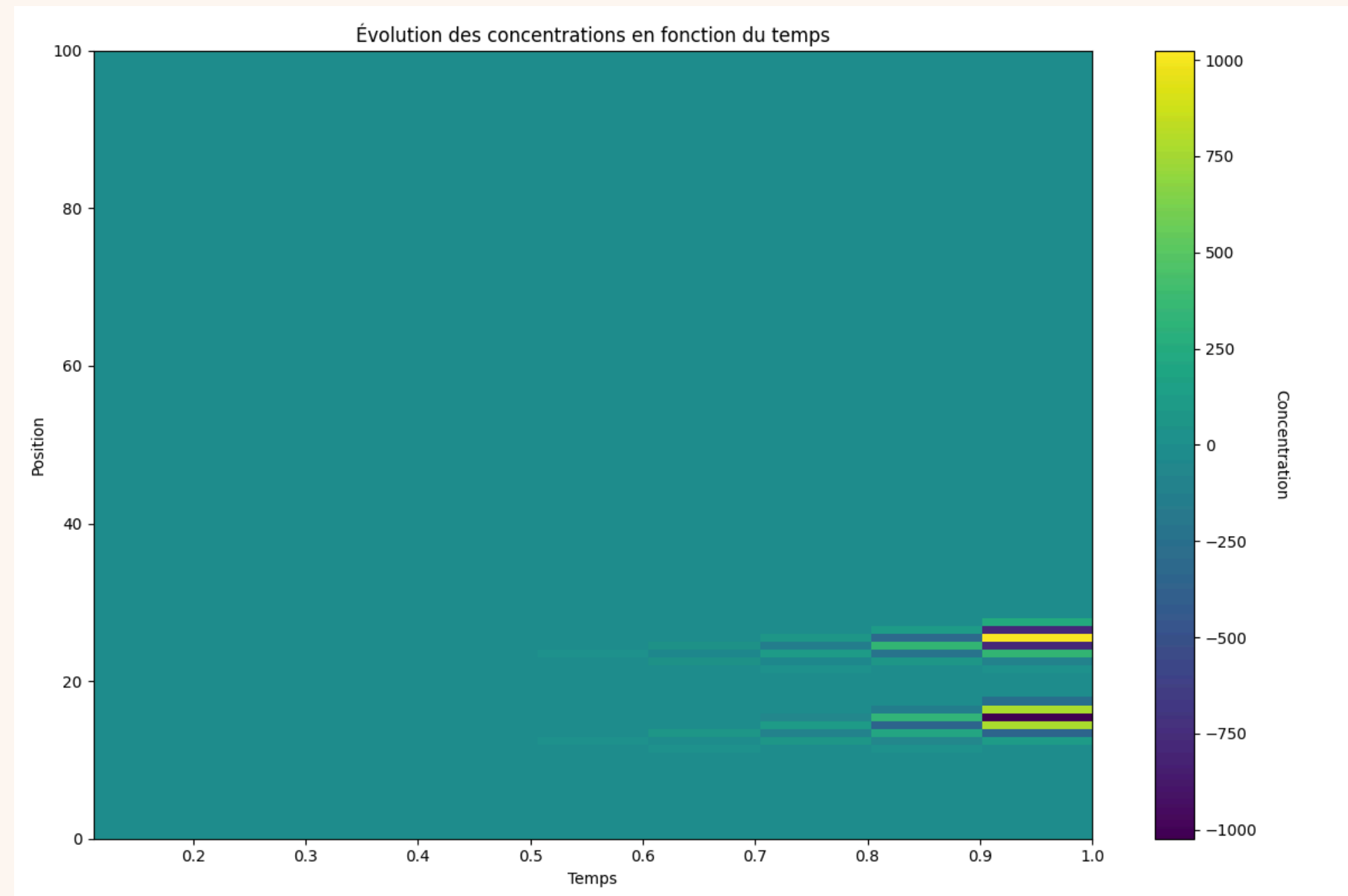
$C_0 = 1$ et $u = 0$



STABILITE

Si en un pas de temps Δt ,
l'information parcourt une
distance $U \cdot \Delta t > \Delta x$, alors :

- L'information "saute" par-dessus des points du maillage
- Le schéma essaie d'extrapoler au lieu d'interpoler



VERIFICATION

ETAPE 3: Validation du programme

La solution exacte du problème : $C(x,t) = C_0 f(x-ut)$

VERIFICATION

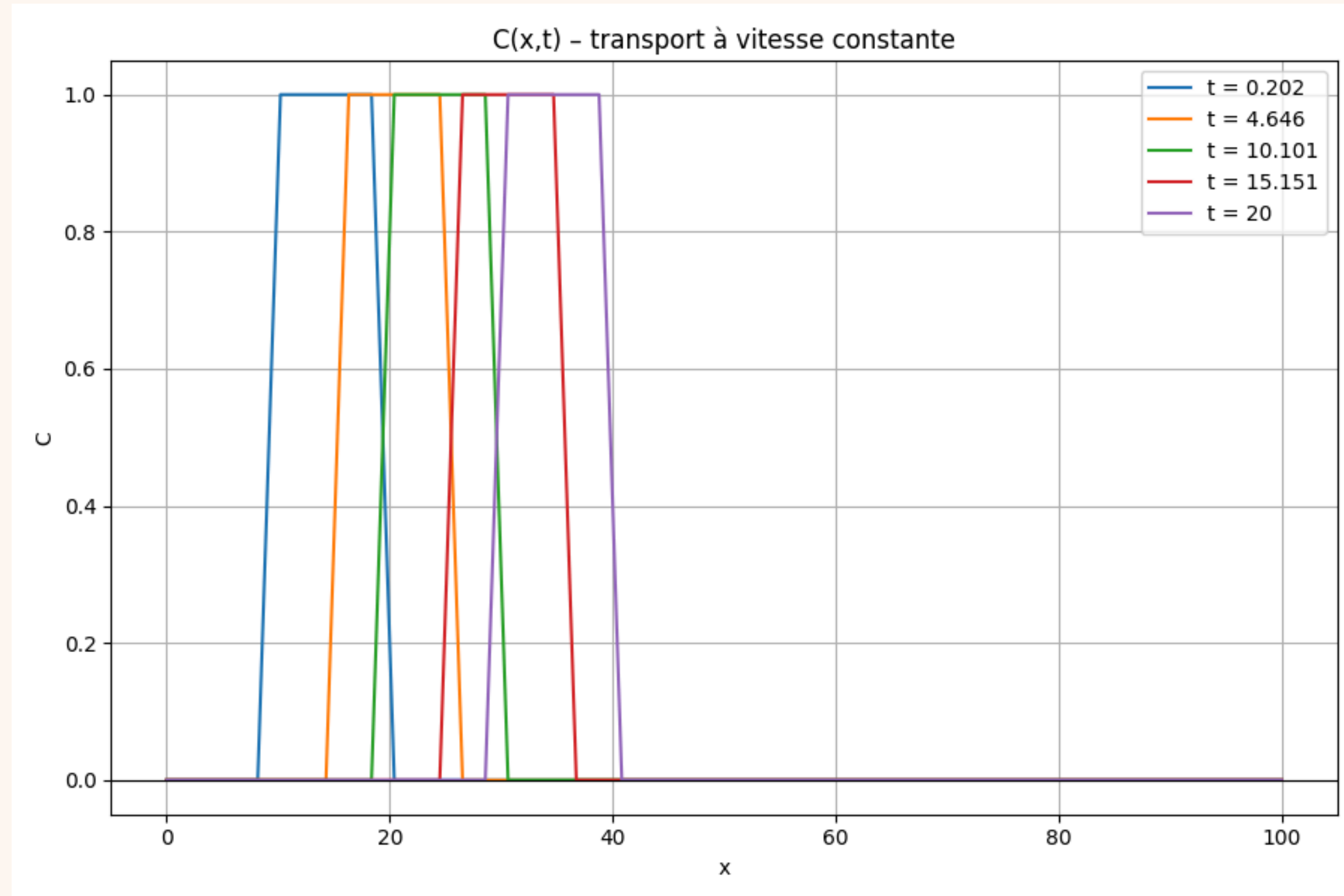
ETAPE 3: Validation du programme

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 L = 100
5 u = 1
6 Nx = 50
7 CO = 1
8 t_valeurs = [0.202, 4.646, 10.101, 15.151, 20]
9
10 xreg = np.linspace(0, L, Nx)
11
12 def H(x):
13     return np.where(x >= 0, 1, 0)
14
15 # Concentration advectée
16 def concentration(L, CO, u, Nx, t, x):
17     # front initial entre L/10 et L/5
18     return CO * (H(x - u*t - L/10) - H(x - u*t - L/5))
19
```

```
19
20 plt.figure(figsize=(9,6))
21
22 for t in t_valeurs:
23     C = concentration(L, CO, u, Nx, t, xreg)
24     plt.plot(xreg, C, label=f"t = {t}")
25
26 plt.axhline(0, linewidth=0.8, color='black')
27 plt.xlabel("x")
28 plt.ylabel("C")
29 plt.title("C(x,t) - transport à vitesse constante")
30 plt.grid(True)
31 plt.legend()
32 plt.tight_layout()
33 plt.show()
```

VERIFICATION

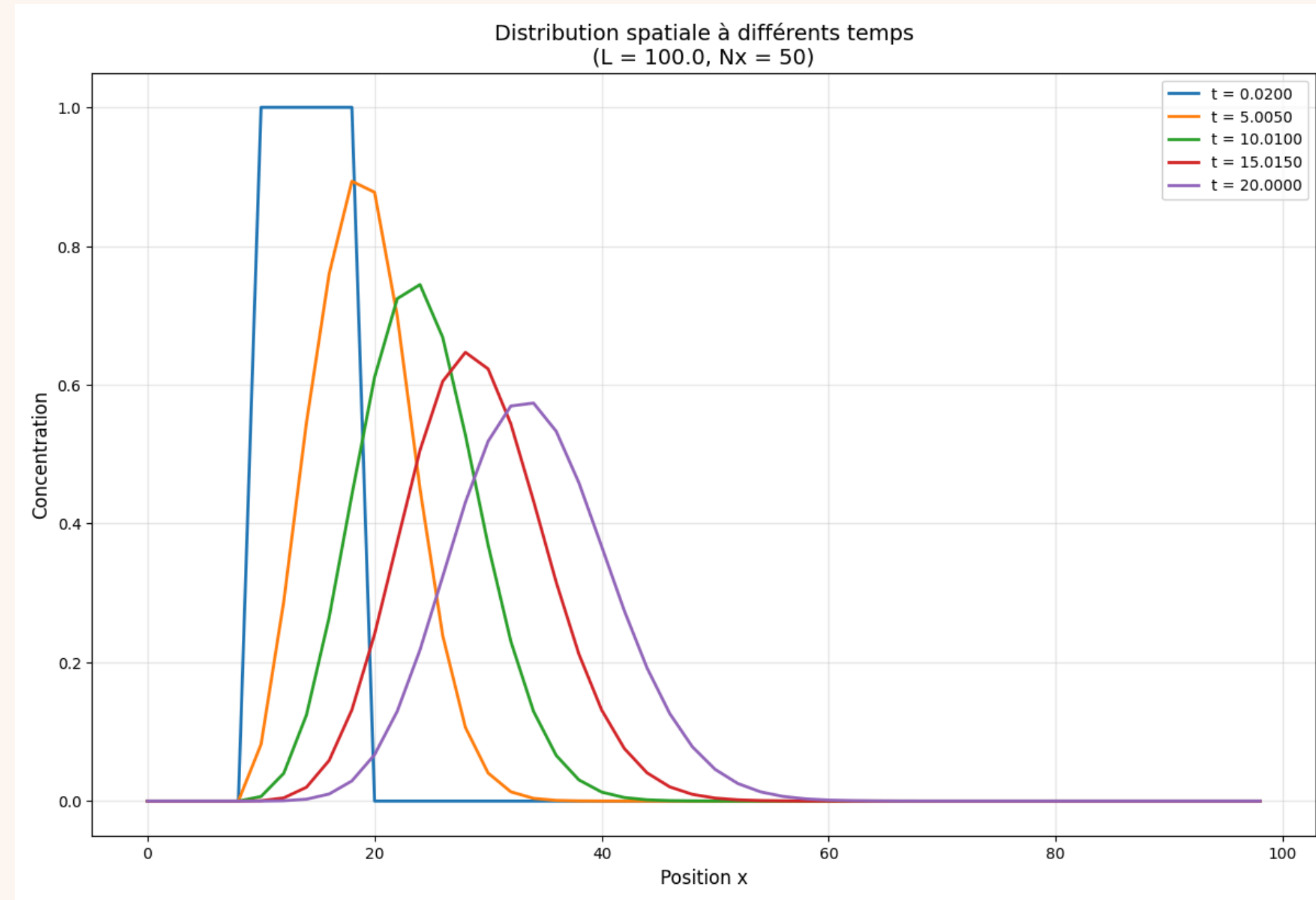
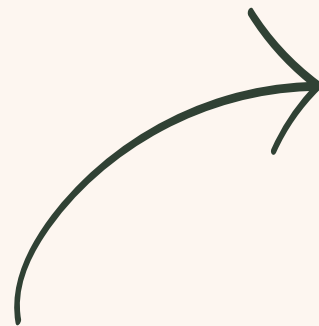
ETAPE 3: Validation du programme



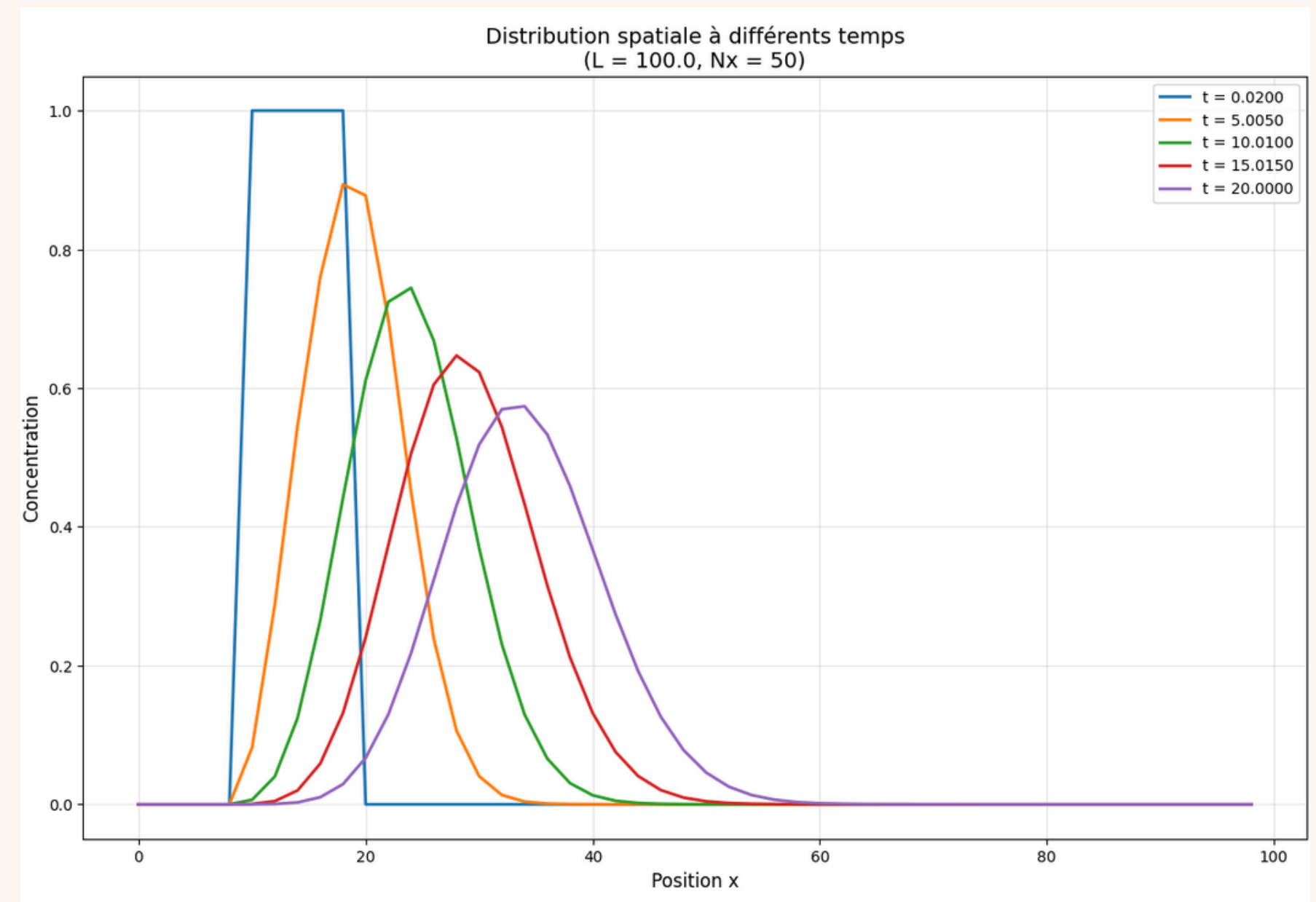
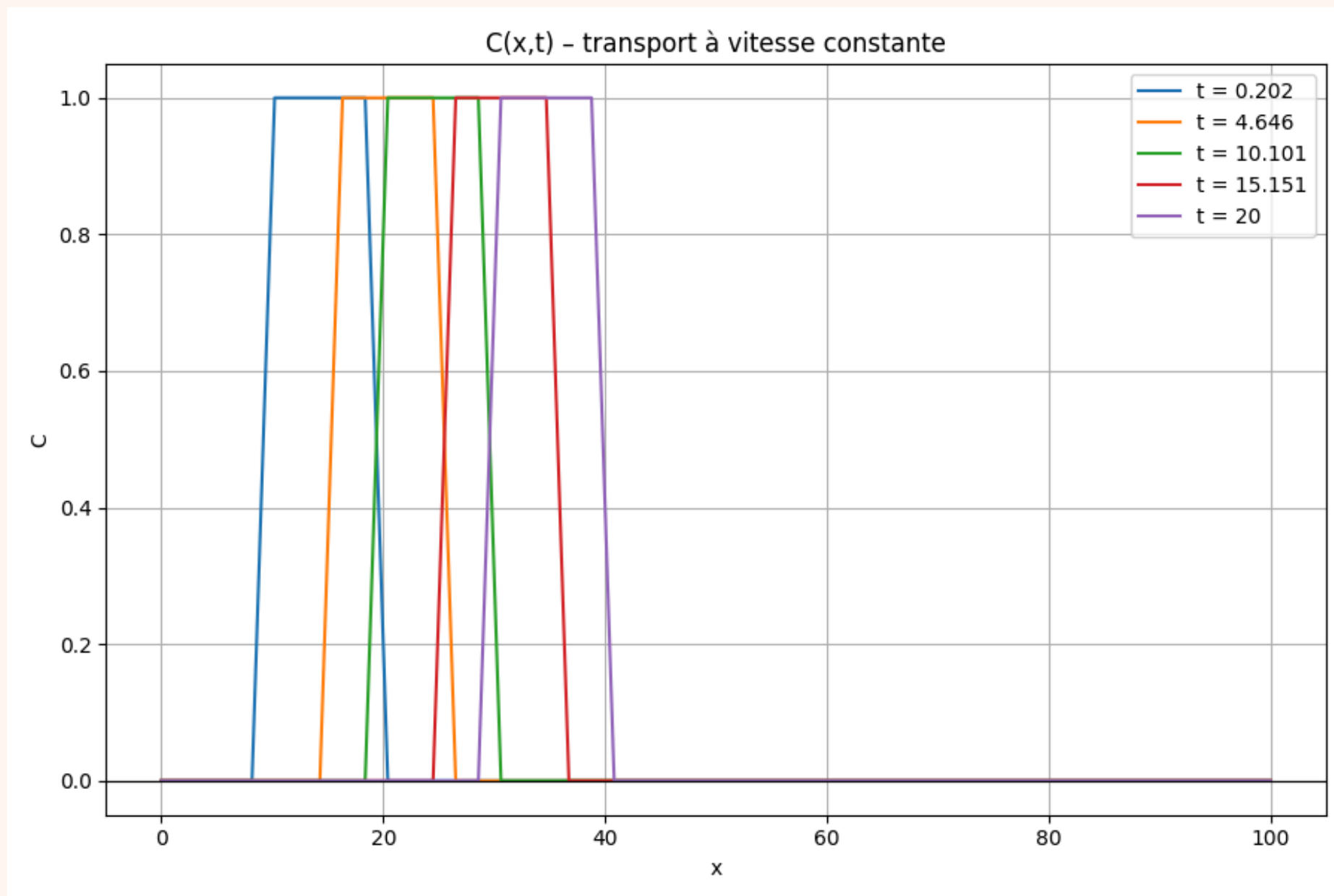
VERIFICATION

ETAPE 3: Validation du programme AVEC NOTRE PROGRAMME

Diffusion
numérique



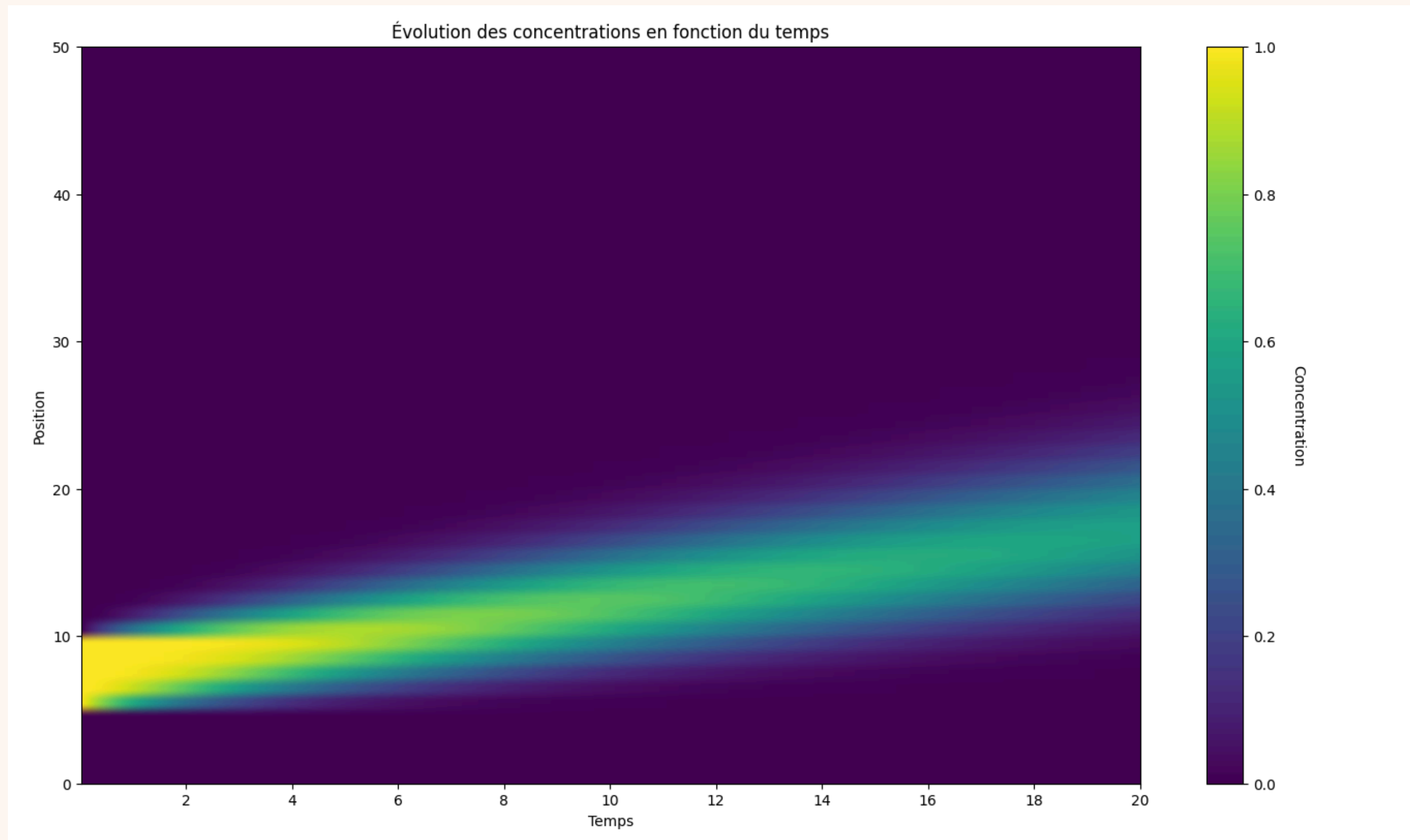
VERIFICATION



VERIFICATION

$C_0 = 1 \text{ mol/L}$

$U = 1 \text{ m/s}$



RAFFINEMENT

ETAPE 5: Raffinement local du maillage

$$x_k^{irreg} = x_k^{reg} + \gamma \frac{L}{3\pi} \sin\left(\frac{2\pi}{L} x_k^{reg}\right) \quad \text{pour } k \in [1, N]$$

$\gamma = -1$ ou $\gamma=1$ ou $\gamma=0$

RAFFINEMENT

ETAPE 5: Raffinement local du maillage

```
SUBROUTINE meshirreg (x_irreg,x_reg,t_reg, get,recup)
  USE m_type
  IMPLICIT NONE
  TYPE(phys), INTENT(IN) :: recup
  TYPE(num), INTENT(IN) :: get

  REAL, DIMENSION(get%Nx), INTENT(INOUT) :: x_reg
  REAL, DIMENSION(get%Nt), INTENT(INOUT) :: t_reg
  REAL, DIMENSION(get%Nx), INTENT(OUT) :: x_irreg
  INTEGER :: i, gam
  REAL :: pi
  CALL mesh(x_reg,t_reg, get,recup)
  gam=1
  pi=acos(real(-1))
  DO i=1,get%Nx
    x_irreg(i) = x_reg(i) + real(gam)*recup%L*sin((2.*pi*x_reg(i))/recup%L)/(3.*pi)
  END DO
END SUBROUTINE meshirreg
```

RAFFINEMENT

```
PROGRAM BE_ETAPE_5
  USE m_type
  IMPLICIT NONE

  TYPE(phys) :: recup
  TYPE(num) :: get
  REAL, DIMENSION (:), ALLOCATABLE :: CT, CDT
  REAL, DIMENSION (:), ALLOCATABLE :: x_reg, t_reg, x_irreg

  CALL read_data(get, recup)

  PRINT *, 'Nx =', get%Nx, 'Nt =', get%Nt
  PRINT *, 'L =', recup%L, 'U =', recup%U, 'C0 =', recup%C0, 'tf =', recup%tf

  IF (get%Nx <= 0 .OR. get%Nt <= 0) THEN
    PRINT *, 'ERREUR: Nx ou Nt <= 0'
    STOP
  END IF

  ALLOCATE(CT(get%Nx), CDT(get%Nx), x_irreg(get%Nx), t_reg(get%Nt), x_reg(get%Nx))
  CT = 0.0
  CDT = 0.0

  CALL mesh(x_reg, t_reg, get, recup)

  CALL meshirreg(x_irreg, x_reg, t_reg, get, recup)

  CALL spat(t_reg, x_irreg, CT, CDT, get, recup)

  DEALLOCATE(CT, CDT, t_reg, x_reg, x_irreg)

END PROGRAM BE_ETAPE_5
```

RAFFINEMENT

ETAPE 5: Raffinement local du maillage

Critère de stabilité respecté si : $\Delta t \leq \min(\Delta x)/u$

$L=100$

$N_x=50$

$N_t=100$

$tf=20$

$Co=1$

$U=1$

